Microtech
INDUSTRIAL SOLUTIONS, INC.
"We go the extra mile..."

Spare Parts Inventory Management System

System Design Document

Status	Draft for review
Date	September 13, 2026
Requirements	SPIMS PRD v1.0
Product Owner	Microtech Industrial Solutions, Inc.

CONFIDENTIAL — FOR MICROTECH USE ONLY

Contents

1. Executive summary 5

2. Scope and assumptions 6

2.1 In scope (v1) 6

2.2 Explicitly deferred (v2+) 6

2.3 Assumptions 6

3. Architecture 7

3.1 Shape 7

3.2 Why Livewire rather than an SPA 7

3.3 Authentication: one identity system, not two 7

3.4 Roles 8

3.5 Two actors, one transaction 8

3.6 Deployment model: one install per company 8

4. Data model 10

4.1 The central invariant 10

4.2 Conventions 10

4.3 Schema 10

4.4 Stock level semantics — resolving an ambiguity in the proposal 15

4.5 What balance_after means, and what backdating does to it 16

4.6 Two distinct audit trails 16

5. Core workflows 17

5.1 Withdrawal — mapped to proposal §4, Steps 1–8 17

5.2 The two details that carry the correctness of this system 17

5.3 Insufficient stock — the override path 17

5.4 Replenishment (proposal §3B) 18

5.5 Reversal and correction 18

5.6 Physical counting (fills the §6 gap) 19

5.7 Excel import (Phase 1 → Phase 4) 20

6. Consumption and depletion analytics 21

6.1 The baseline formula is the proposal's own 21

6.2 Definitions that must be stated or reports will disagree with each other 21

6.3 Data confidence 21

6.4 Movement classification 21

6.5 Query strategy 22

6.6 Predicted out-of-stock 22

7. Alerts (proposal §6) 23

7.1 Alerts are state-based, not event-based 23

7.2 Evaluation and delivery 23

8. Security and operational risk 24

8.1 RFID provides accountability, not prevention 24

8.2 Badge storage 24

8.3 Application security baseline 24

8.4 Availability: online-only against a 24/7 storeroom 24

9. Reports (proposal §7 A–H) 25

9.1 Report I is not in §7 — and needs to be 25
9.2 Dashboard (§3B, "Dashboard/summary view") 25
10. Hardware integration boundary 26
11. Supabase free tier: constraints and mitigations 27
12. Non-functional requirements 28
12.1 Laravel + Supabase connection configuration 28
12.2 Generated columns in Laravel migrations 28
13. Testing strategy 29
13.1 The invariant test 29
13.2 Concurrency test 29
13.3 Scheduled verification in production 29
13.4 Remaining coverage 29
13.5 Install tests 29
14. Delivery phases (mapped to proposal §11) 30
15. Traceability — every proposal requirement 31
15.1 Additions beyond the proposal — require client confirmation 31
16. Open questions for the client 33
17. Decisions recorded 34

Product owner	Microtech Industrial Solutions, Inc.
First installs	Microtech production storeroom · Microtech demo
Deployment model	One install per company — separate application and database (§3.6)
Source proposal	*Project Proposal: Spare Parts Inventory Management System*, Engr. Jabez J. Agaya — prepared by Microtech for a customer
Requirements	PRD: `docs/prd/Microtech_SPIMS_PRD_v1.0.md`
Current focus	Microtech production and demo installs. Customer installs parked until customer details are confirmed
Document status	Draft for review
Date	2026-09-12
Stack	Laravel 12 (PHP 8.3) · Livewire 3 · Tailwind · Supabase Postgres (free tier)
Classification	Confidential — for Microtech use only

1. Executive summary

This document translates the spare parts proposal Microtech prepared for a customer into a buildable system design. SPIMS is a **Microtech-owned product**: built once and installed separately for each company, starting with Microtech's own storeroom and a demo install used to show it to customers (§3.6). Each install is a web-hosted application operated by a 24/7 Inventory Custodian, in which every movement of stock is recorded as an immutable ledger entry attributed to both the custodian who processed it and the personnel who received the parts.

Three readings of the proposal shape the entire architecture:

1. **The personnel ID is scanned last, not first** (§3C, §4 Step 5). The scan is not a login — it is the commit signature on a completed transaction. A withdrawal is therefore a *draft cart that a scan commits*, and every transaction has **two distinct actors**: the custodian (an authenticated system user, logged in for the whole shift) and the requesting personnel (identified by badge, who never logs in).
2. **The audit trail sample in §3B carries a running `Inventory Balance` column.** That is a ledger, not a log. Stock is therefore *derived* from immutable signed movement rows, each stamping the balance that resulted from it. Mutating a `current_quantity` column in place would make the audit trail unverifiable and the Inventory Discrepancy Alert (§6) impossible to compute honestly.
3. **§6 promises an Inventory Discrepancy Alert that no section of the proposal defines a process for.** §3A lists "conducting periodic physical inventory counts" as a custodian duty, but there is no counting workflow, no variance handling, and no approval path. This design fills that gap with a cycle-count subsystem. **This is an addition beyond the proposal and should be confirmed with the client.**

2. Scope and assumptions

2.1 In scope (v1)

Part master and storage locations · equipment register · personnel and badge register · multi-line withdrawal with scan-last commit · replenishment receipts · stock adjustments · physical/cycle counting with variance posting · Excel/CSV part and opening-balance import · consumption and depletion analytics · six alert types (the proposal's five plus predicted out-of-stock) · the eight reports of proposal §7 plus Replenishment (I), Exceptions (J) and Predicted Out-of-Stock (K) · role-based access control · dual audit trail (stock ledger + configuration changes) · paper-fallback backdated entry · nightly off-platform database backup · per-install settings (company profile, time zone, currency, withdrawal purposes, transaction prefixes, thresholds) · sample data loading for demo installs.

2.2 Explicitly deferred (v2+)

Offline-capable PWA station · lead-time-aware reorder point and statistical safety stock · multi-location stock per part · manufacturer/alternate part numbers (`part_aliases`) · inventory valuation reporting · purchase order management · supplier master · barcode labelling of parts themselves · mobile app · return-to-stock of unused parts as its own transaction type (interim: reverse and re-record; PRD Appendix A Q13 decides whether this moves into v1).

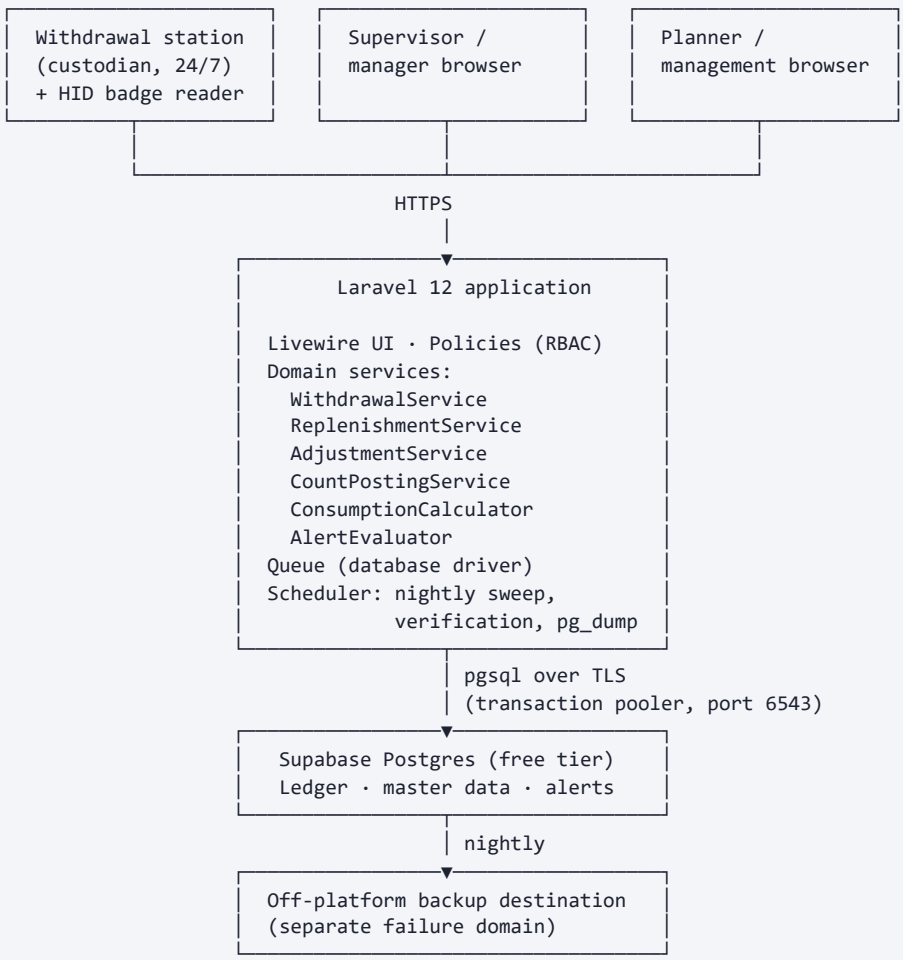
2.3 Assumptions

Assumption	Basis
Single-tenant per install; one install per company	Product decision 2026-09-13; see §3.6
One withdrawal station initially, designed for more	Client answer; §3B says "a dedicated computer workstation"
HID keyboard-wedge ID reader	Hardware not yet purchased; see §10
Existing part data arrives as Excel/CSV	Client answer; drives importer into v1
One storage location per part	§3B lists "storage location" as a part attribute
Time zone is an install setting, default Asia/Manila; all timestamps stored `timestamptz` in UTC	Philippine operation first; other companies may differ
Quantities are decimal, not integer	Bearings are whole units; oil, cable and wire are not

3. Architecture

3.1 Shape

A Laravel monolith serving server-rendered Livewire components, backed by Supabase's managed Postgres. One codebase, one deployment, one authentication system.



3.2 Why Livewire rather than an SPA

The withdrawal cart needs live reactivity — add a part number, immediately show its description, on-hand quantity and resulting balance — without a page reload. Livewire delivers that with no separate frontend build, no API surface to secure twice, and no second authentication model. An Inertia/Vue SPA would be the right call if a native mobile app or offline PWA were a near-term requirement; they are deferred (§2.2), so that flexibility would be pre-paid for no current return. If offline stations later become a hard requirement, that is the migration path.

3.3 Authentication: one identity system, not two

Laravel Fortify owns authentication. Supabase Auth is not used.

Supabase Auth earns its place when browsers speak to PostgREST directly and RLS is the authorization boundary. Here, Laravel connects as a privileged Postgres role and enforces authorization in its own policies. Adding Supabase Auth would introduce a second identity store and a permanent JWT-to-Eloquent-user mapping layer while adding no control the application does not already have.

Supabase's role in this system is managed Postgres, plus Storage for generated report files and part photographs.

RLS posture. Every table nonetheless gets `ENABLE ROW LEVEL SECURITY` with deny-all policies and no grants to `anon` or `authenticated`. Laravel's owner role bypasses these, so they cost nothing operationally — but if the project's anon key is ever exposed, it yields nothing. Defence in depth for roughly twenty lines of migration.

3.4 Roles

Role	Capabilities
`custodian`	Process withdrawals, replenishments; perform counts; view stock and own transactions. **Cannot** edit min/max levels, void transactions, or post count variances.
`supervisor`	All custodian rights, plus: authorise stock-correction overrides, void/reverse transactions, post count variances, manage personnel and badges.
`planner`	Read-only on transactions; full analytics and reports; owns min/max, reorder point, critical level and criticality classification.
`admin`	User management, system settings, imports (including opening balances), full badge identifiers. **Does not** process stock transactions — a user needing both holds both roles, so IT administration and stock handling stay separate.

Implemented with `spatie/laravel-permission` plus Laravel policies. Four roles do not justify a hand-rolled system when a battle-tested one exists.

3.5 Two actors, one transaction

This distinction runs through the entire data model and must not be collapsed:

	System users (`users`)	**Personnel (`personnel`)**
Who	Custodian, supervisor, planner, admin	Maintenance technicians who withdraw parts
Authenticates by	Email + password (Laravel session)	Badge scan, at commit time only
Logs in?	Yes, for a whole shift	**Never**
Role in a withdrawal	Processes it (`custodian_user_id`)	Receives the parts (`personnel_id`)

3.6 Deployment model: one install per company

SPIMS is a Microtech product. It is built once and **installed separately for each company**: every install is its own Laravel application instance with its own Postgres database, file storage, users and settings. Microtech runs the first two installs — **production**, for its own storeroom, and **demo**, holding sample data for showing SPIMS to customers.

Why not one shared multi-company system. A shared system puts `company_id` on every row and every query, and a single missed filter in a report or export shows one company's stock and personnel to another. Large industrial customers' IT and security teams commonly refuse shared hosting of their data regardless, and some will require installs on their own infrastructure. Retrofitting shared hosting later means rewriting the data layer, so the decision is recorded here: **the schema has no company scoping, deliberately.**

Five rules follow, and each constrains the code, not just the deployment:

1. **No company-specific values in code.** Company name and logo, time zone, currency, withdrawal purposes, transaction number prefixes and thresholds live in `settings` and `withdrawal_purposes` (§4.3). The proposal's purpose list is seed data, not a `CHECK` constraint.
2. **Plain Postgres only.** No Supabase Auth, and no application logic that depends on RLS, Realtime, Edge Functions or PostgREST. Supabase is a Postgres host for Microtech's installs; a customer install must run unchanged on RDS, Cloud SQL, Azure Database or on-premises Postgres.
3. **Files through Laravel's filesystem abstraction.** Photos and exports go to a configured disk — S3-compatible (Supabase Storage, AWS S3, MinIO) or local — chosen by environment configuration.
4. **One codebase, one release line.** Every install runs a tagged release; migrations run per install. No per-customer branches. A customer request becomes a product feature for every install (behind a setting if it must differ) or is

declined. Forked installs cannot be upgraded together and quickly become separate products to maintain.

5. **Demo and production are separate installs.** `APP_INSTALL_TYPE=demo|production` is set in environment configuration — not in the database, not in the UI. The sample-data seeder refuses to run unless it is `demo`, and a demo install shows a banner on every screen and export.

Provisioning a new install — the repeatable checklist run for every company, and what the fresh-install test in §13.5 proves:

1. Create the Postgres database and file storage; set environment configuration
2. Run migrations against the empty database
3. Seed default settings and the default withdrawal purpose list
4. Create the first System Administrator
5. Set company profile, time zone and currency
6. Create storage locations and the equipment register
7. Import spare parts, then opening balances — dry run first
8. Register personnel and enroll badges
9. Configure the nightly off-platform backup and complete one test restore

4. Data model

4.1 The central invariant

> **For every part:** `part_stock.on_hand == SUM(stock_movements.quantity)`.

`stock_movements` is the source of truth and is append-only. `part_stock` is a transactionally maintained cache that exists to make reads fast and to serve as the row-lock target. It must always be reproducible by replaying the ledger, and a scheduled verification job proves this nightly (§13.3).

Three consequences designed in deliberately:

- **Opening balances from the Excel import are written as `opening` movements**, never as a direct `on_hand` write. The ledger is complete from day zero and the invariant holds from the first row.
- **Nothing is ever deleted.** A mistaken transaction is corrected by a **reversing transaction** linked to the original via `reverses_transaction_id`, requiring supervisor role and a reason.
- **`occurred_at` and `recorded_at` are separate columns from day one.** Two columns now make backdated paper-fallback entries reconcilable forever (§8.4).

4.2 Conventions

Per Supabase Postgres guidance: `bigint generated always as identity` primary keys (sequential, no index fragmentation); `text` rather than `varchar(n)`; `timestamptz` never bare `timestamp`; `numeric` for all quantities and money; `text` + `CHECK` rather than Postgres enums (enum values cannot be altered transactionally); foreign key columns used in joins, filters and lookups explicitly indexed. User-reference audit columns such as `reversed_by_user_id` are not indexed — users are never deleted and those columns are not queried by.

4.3 Schema

```
create extension if not exists pg_trgm;

-- ─────────────────────────── Master data ──────────────────────────

create table storage_locations (
  id          bigint generated always as identity primary key,
  code        text not null unique,
  name        text not null,
  description text,
  is_active   boolean not null default true,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

create table equipment (
  id         bigint generated always as identity primary key,
  code       text not null unique,
  name       text not null,
  area       text,
  is_active  boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table parts (
  id                  bigint generated always as identity primary key,
  part_number         text not null,
  description         text not null,
  unit_of_measure     text not null default 'pcs',
  default_location_id bigint references storage_locations(id),
  min_level           numeric(14,3) not null default 0,
  max_level           numeric(14,3),
```

```
  reorder_point       numeric(14,3) not null default 0,
  critical_level      numeric(14,3) not null default 0,
  criticality         text not null default 'normal'
                        check (criticality in ('critical','important','normal')),
  lead_time_days      integer,          -- captured now, used by the v2 reorder model
  is_active           boolean not null default true,
  created_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now(),
  constraint parts_levels_ordered check (
    critical_level <= min_level
    and (max_level is null or max_level >= min_level)
  ),
  constraint parts_levels_non_negative check (
    min_level >= 0 and reorder_point >= 0 and critical_level >= 0
  )
);

create unique index parts_part_number_key   on parts (lower(part_number));
create index parts_description_trgm         on parts using gin (description gin_trgm_ops);
create index parts_criticality_active_idx   on parts (criticality) where is_active;
create index parts_location_idx             on parts (default_location_id);

-- Proposal §3B lists "Equipment/application" as a part-master field, and §11 Phase 1
-- requires parts to be "classified according to equipment". A part commonly fits more
-- than one machine, so this is a many-to-many association, not a column on parts.
-- It answers "which parts fit machine X?" at the counter, and is distinct from
-- stock_transactions.equipment_id, which records where a part was ACTUALLY used.
create table part_equipment (
  part_id      bigint not null references parts(id) on delete cascade,
  equipment_id bigint not null references equipment(id) on delete cascade,
  notes        text,
  primary key (part_id, equipment_id)
);

create index part_equipment_equipment_idx on part_equipment (equipment_id);

-- Derived stock cache and lock target. One row per part.
create table part_stock (
  part_id    bigint primary key references parts(id) on delete restrict,
  on_hand    numeric(14,3) not null default 0,
  updated_at timestamptz not null default now(),
  constraint part_stock_non_negative check (on_hand >= 0)
);

-- Requestors. Never authenticate into the application.
create table personnel (
  id                  bigint generated always as identity primary key,
  employee_no         text not null unique,
  full_name           text not null,
  department          text,
  badge_uid_hash      text unique,   -- sha256(uid || pepper) — the scan lookup key
  badge_uid_encrypted text,          -- Laravel encrypted cast, admin-visible only
  badge_uid_masked    text,          -- display form, e.g. A1234**** as in the proposal §3B sample
  photo_path          text,          -- Supabase Storage key; shown at scan for visual confirm
  is_active           boolean not null default true,
  created_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now()
);

-- Withdrawal purposes differ between companies, so they are rows, not a CHECK constraint (§3.6).
-- Seeded in every install with the proposal's list: Preventive Maintenance, Corrective
-- Maintenance, Breakdown Repair, Machine Modification, Replacement of Worn Part, and
-- Other Maintenance Activity (requires_remarks = true).
create table withdrawal_purposes (
  id               bigint generated always as identity primary key,
  code             text not null unique,     -- stable identifier; never changed once used
  name             text not null unique,     -- display name; may be renamed
  requires_remarks boolean not null default false,
  sort_order       integer not null default 0,
  is_active        boolean not null default true,
  created_at       timestamptz not null default now(),
```

```
  updated_at       timestamptz not null default now()
);

-- ─────────────────────── Transactions and ledger ──────────────────────

create table stock_transactions (
  id                      bigint generated always as identity primary key,
  transaction_no          text not null unique,          -- WD-2026-000123 / RP- / AJ- / OP-
  type                    text not null
                            check (type in ('withdrawal','replenishment','adjustment','opening')),
  -- Status vocabulary follows the PRD: draft -> completed | cancelled,
  -- completed -> reversed. There is no pending/approved step; exceptions are authorised
  -- while the transaction is still a draft.
  status                  text not null default 'draft'
                            check (status in ('draft','completed','cancelled','reversed')),
  custodian_user_id       bigint not null references users(id),
  personnel_id            bigint references personnel(id),
  equipment_id            bigint references equipment(id),
  purpose_id              bigint references withdrawal_purposes(id),
  supplier                text,
  reference_no            text,        -- DR / PO number for replenishment
  reason_code             text check (reason_code in ('miscount','damage','unrecorded_issue',
                            'unrecorded_receipt','found_stock','shrinkage','other')),
  remarks                 text,
  occurred_at             timestamptz not null,
  recorded_at             timestamptz,
  is_backdated            boolean not null default false,
  backdate_reason         text,
  reverses_transaction_id bigint references stock_transactions(id),
  reversed_by_user_id     bigint references users(id),
  reversal_reason         text,
  -- How the requesting personnel was identified (PRD FR-SPIMS-007, FR-SPIMS-012).
  identification_method   text check (identification_method in
                            ('badge_scan','supervisor_manual','paper_form')),
  identification_authorised_by_user_id bigint references users(id),
  identification_reason   text,
  paper_form_no           text unique,  -- backdated entries only; unique blocks double entry
  created_at              timestamptz not null default now(),
  updated_at              timestamptz not null default now(),

  -- A completed withdrawal must be fully attributed. This is the accountability
  -- guarantee of proposal §2 expressed as a database constraint.
  -- Reversing transactions are exempt: they copy personnel/equipment/purpose from the
  -- original for reporting, but nobody was identified at the counter when they were made.
  constraint completed_withdrawal_is_attributed check (
    not (status = 'completed' and type = 'withdrawal' and reverses_transaction_id is null)
    or (personnel_id is not null and equipment_id is not null and purpose_id is not null
        and identification_method is not null)
  ),
  constraint completed_has_recorded_at check (
    status not in ('completed','reversed') or recorded_at is not null
  ),
  -- Manual identification is an exception: it must name the supervisor and say why.
  constraint manual_identification_is_authorised check (
    identification_method is distinct from 'supervisor_manual'
    or (identification_authorised_by_user_id is not null and identification_reason is not null)
  ),
  constraint backdated_has_paper_form check (
    not is_backdated or paper_form_no is not null
  ),
  constraint backdated_has_reason check (
    not is_backdated or backdate_reason is not null
  ),
  -- An adjustment moves stock without a physical issue or receipt. It must always
  -- say why, or the ledger accumulates unexplained corrections.
  constraint adjustment_has_reason check (
    type <> 'adjustment' or status <> 'completed' or reason_code is not null
  )
);

create index stock_transactions_status_type_idx on stock_transactions (status, type);
```

```
create index stock_transactions_occurred_idx    on stock_transactions (occurred_at desc);
create index stock_transactions_personnel_idx   on stock_transactions (personnel_id);
create index stock_transactions_equipment_idx   on stock_transactions (equipment_id);
create index stock_transactions_custodian_idx   on stock_transactions (custodian_user_id);
create index stock_transactions_purpose_idx     on stock_transactions (purpose_id);

-- Indexes the reversal lookup and enforces "a transaction can be reversed only once".
create unique index stock_transactions_reverses_key
  on stock_transactions (reverses_transaction_id)
  where reverses_transaction_id is not null;

create table stock_transaction_lines (
  id                  bigint generated always as identity primary key,
  stock_transaction_id bigint not null references stock_transactions(id) on delete cascade,
  line_no             integer not null,
  part_id             bigint not null references parts(id),
  quantity            numeric(14,3) not null check (quantity > 0),
  equipment_id        bigint references equipment(id),   -- per-line override of the header
  purpose_id          bigint references withdrawal_purposes(id), -- per-line override of the header
  unit_cost           numeric(14,4),                     -- replenishment only
  remarks             text,
  unique (stock_transaction_id, line_no),
  unique (stock_transaction_id, part_id)   -- one line per part; the UI merges duplicates
);

create index stock_transaction_lines_part_idx on stock_transaction_lines (part_id);

-- THE LEDGER. Append-only. Signed quantities. Never updated, never deleted.
create table stock_movements (
  id                      bigint generated always as identity primary key,
  part_id                 bigint not null references parts(id),
  stock_transaction_id    bigint not null references stock_transactions(id),
  stock_transaction_line_id bigint references stock_transaction_lines(id),
  movement_type           text not null check (movement_type in
                            ('withdrawal','replenishment','adjustment','opening','count_variance')),
  quantity                numeric(14,3) not null check (quantity <> 0),  -- SIGNED: -15 / +35
  balance_after           numeric(14,3) not null,
  occurred_at             timestamptz not null,
  recorded_at             timestamptz not null default now(),
  personnel_id            bigint references personnel(id),
  equipment_id            bigint references equipment(id),
  purpose_id              bigint references withdrawal_purposes(id),
  custodian_user_id       bigint not null references users(id),
  created_at              timestamptz not null default now()
);

create index stock_movements_part_occurred_idx on stock_movements (part_id, occurred_at desc);
create index stock_movements_occurred_idx      on stock_movements (occurred_at desc);
create index stock_movements_personnel_idx     on stock_movements (personnel_id, occurred_at desc);
create index stock_movements_equipment_idx     on stock_movements (equipment_id, occurred_at desc);
create index stock_movements_txn_idx           on stock_movements (stock_transaction_id);
create index stock_movements_purpose_idx       on stock_movements (purpose_id);

-- Every consumption, depletion and monthly report filters on withdrawals only.
-- A partial index keeps that hot path off the replenishment and adjustment rows.
create index stock_movements_withdrawals_idx
  on stock_movements (part_id, occurred_at)
  where movement_type = 'withdrawal';

-- Enforce append-only at the database, not merely by convention in the application.
create or replace function prevent_ledger_mutation() returns trigger
language plpgsql as $fn$
begin
  raise exception 'stock_movements is append-only (attempted % on id %)', tg_op, old.id;
end;
$fn$;

create trigger stock_movements_immutable
  before update or delete on stock_movements
  for each row execute function prevent_ledger_mutation();
```

```
-- ──────────────────── Physical counting (fills the §6 gap) ────────────────────

create table count_sessions (
  id                     bigint generated always as identity primary key,
  session_no             text not null unique,
  scope                  text not null check (scope in ('full','location','criticality','adhoc')),
  scope_value            text,
  status                 text not null default 'open'
                           check (status in ('open','counted','posted','cancelled')),
  variance_tolerance_pct numeric(6,3) not null default 0,
  opened_by_user_id      bigint not null references users(id),
  opened_at              timestamptz not null default now(),
  posted_by_user_id      bigint references users(id),
  posted_at              timestamptz,
  notes                  text
);

create table count_lines (
  id                bigint generated always as identity primary key,
  count_session_id  bigint not null references count_sessions(id) on delete cascade,
  part_id           bigint not null references parts(id),
  system_qty        numeric(14,3) not null,     -- snapshot when the session opened
  counted_qty       numeric(14,3) check (counted_qty is null or counted_qty >= 0),
  counted_at        timestamptz,                -- when the shelf was physically counted

  -- Drift between session open and the physical count. FOR ANALYSIS ONLY.
  -- This is NOT the quantity posted to the ledger — see posting_delta and §5.6.
  snapshot_variance numeric(14,3) generated always as (counted_qty - system_qty) stored,

  on_hand_at_post   numeric(14,3),              -- live balance read under lock at posting
  posting_delta     numeric(14,3),              -- counted_qty - on_hand_at_post; the movement qty

  reason_code       text check (reason_code in ('miscount','damage','unrecorded_issue',
                       'unrecorded_receipt','found_stock','shrinkage','other')),
  notes             text,
  unique (count_session_id, part_id)
);

create index count_lines_part_idx on count_lines (part_id);

-- ──────────────────────────── Alerts ────────────────────────────

create table alerts (
  id                     bigint generated always as identity primary key,
  -- NOT NULL is load-bearing: Postgres treats NULLs as distinct in a unique index,
  -- so a nullable part_id would silently defeat the open-once guarantee below.
  part_id                bigint not null references parts(id),
  type                   text not null check (type in ('low_stock','critical_stock',
                           'replenishment','high_consumption','inventory_discrepancy','predicted_stockout')),
  status                 text not null default 'open'
                           check (status in ('open','acknowledged','resolved')),
  severity               text not null default 'warning'
                           check (severity in ('info','warning','critical')),
  context                jsonb not null default '{}',
  opened_at              timestamptz not null default now(),
  acknowledged_by_user_id bigint references users(id),
  acknowledged_at        timestamptz,
  resolved_at            timestamptz
);

-- This partial unique index IS the open-once mechanism. An alert cannot be raised
-- twice for the same part and condition while one is still outstanding, so the
-- evaluator can run on every commit without ever producing duplicate noise.
create unique index alerts_one_open_per_part_type
  on alerts (part_id, type)
  where status in ('open','acknowledged');

create index alerts_status_idx on alerts (status, opened_at desc);

-- ──────────────── Configuration audit trail and settings ────────────────
```

```
create table audit_logs (
  id             bigint generated always as identity primary key,
  user_id        bigint references users(id),
  auditable_type text not null,
  auditable_id   bigint not null,
  event          text not null,   -- created | updated | deleted | reversed | override_authorised |
manual_identification_authorised | badge_scan_rejected
  old_values     jsonb,
  new_values     jsonb,
  ip_address     inet,
  user_agent     text,
  created_at     timestamptz not null default now()
);

create index audit_logs_auditable_idx on audit_logs (auditable_type, auditable_id, created_at desc);
create index audit_logs_user_idx      on audit_logs (user_id, created_at desc);

-- Per-install configuration (§3.6). Keys: company.name, company.logo_path, company.address,
-- regional.timezone, regional.currency, numbering.prefixes, analysis.window_months,
-- movement_class.*, alerts.high_consumption.*, withdrawal.identification_timeout_seconds,
-- backdating.limit_days, counting.default_tolerance_pct, notifications.*,
-- prediction.default_lead_time_days, prediction.warning_buffer_days
-- The install type (demo | production) is deliberately NOT here: it is environment
-- configuration, so it cannot be changed by anyone with database or admin UI access.
create table settings (
  key                text primary key,
  value              jsonb not null,
  updated_by_user_id bigint references users(id),
  updated_at         timestamptz not null default now()
);

create table import_batches (
  id                 bigint generated always as identity primary key,
  filename           text not null,
  storage_path       text,
  status             text not null default 'pending'
                       check (status in ('pending','validated','completed','failed','cancelled')),
  rows_total         integer not null default 0,
  rows_created       integer not null default 0,
  rows_updated       integer not null default 0,
  rows_failed        integer not null default 0,
  errors             jsonb not null default '[]',
  imported_by_user_id bigint not null references users(id),
  created_at         timestamptz not null default now(),
  completed_at       timestamptz
);
```

4.4 Stock level semantics — resolving an ambiguity in the proposal

§3B lists "Minimum stock level", "Maximum stock level" and "Reorder point" as three separate fields, and §5 asks the system to establish "Minimum Stock Level, Maximum Stock Level, Safety Stock, Reorder Point". In many storerooms "minimum" and "reorder point" are used interchangeably, and if that is left unstated the Low Stock Alert and the Replenishment Alert will fire on the same condition and one of them becomes noise.

The definitions this system uses, to be confirmed with the client:

Field	Meaning	Drives
`reorder_point`	Order *now* to arrive before stock runs out. The **highest** of the three.	Replenishment Alert (§7)
`min_level`	Safety stock. Reaching it means replenishment is already late.	Low Stock Alert (§7)
`critical_level`	Danger threshold — downtime risk is imminent. The **lowest**.	Critical Stock Alert (§7)
`max_level`	Ceiling for replenishment quantity; excess capital above it.	Recommended order quantity

Expected ordering is `critical_level <= min_level <= reorder_point <= max_level`. The database enforces only `critical_level <= min_level` and `max_level >= min_level` as hard constraints; `reorder_point` is deliberately left unconstrained relative to `min_level` because a client may legitimately set them equal, and a hard constraint would block a valid configuration. The part editor warns when the conventional ordering is violated rather than refusing to save.

4.5 What `balance_after` means, and what backdating does to it

The ledger has two time axes and they do not always agree: `occurred_at` (when stock physically moved) and `id` / `recorded_at` (when the system was told). Backdated paper-fallback entries (§8.4) are exactly the case where they diverge, so the meaning of the balance column has to be pinned down or the §13.1 invariant test is untestable.

The canonical order of the ledger is `id` — recording order. `balance_after` is the running sum of `quantity` in `id` order, and nothing ever rewrites an already-stamped value.

Consequences, all of them intended:

- A backdated entry **appends at the end** of the ledger with the balance current at the moment it was recorded. Its `occurred_at` is earlier, and that is honest: the parts moved in the past, the system learned of it later.
- **Consumption analytics filter on `occurred_at`** (§6.2), so a backdated entry does retroactively correct August's consumption figures — which is the desired behaviour.
- **Balance history is reconstructed in `id` order**, so a backdated entry does *not* retroactively rewrite the balances stamped on rows recorded before it. This is the same treatment a late journal entry receives in accounting: the correction is visible as its own line, not smeared backwards through the history.
- Point-in-time balance for a past instant is therefore only exact up to the backdating that had been recorded by then. This is an inherent property of late entry, not a defect — and it is another reason backdated entries carry `is_backdated` and a mandatory reason, and are reported on.

4.6 Two distinct audit trails

The proposal's §3B describes an "audit trail" as "a history/log of all actions and transactions." That is genuinely two different things, and conflating them produces a table that serves neither:

	Stock ledger (`stock_movements`)	**Configuration audit (`audit_logs`)**
Records	Every quantity change, with resulting balance	Who changed a min level, reversed a transaction, edited a part, authorised an override
Shape	Append-only, signed, balance-stamped	Append-only, old/new value diff
Answers	"What is this part's history and is the balance right?"	"Who raised the reorder point last March, and why?"
Matches	The §3B sample table exactly	The §3B phrase "all actions"

5. Core workflows

5.1 Withdrawal — mapped to proposal §4, Steps 1–8

```
Step 1  Personnel requests parts                      (verbal; no system action)
Step 2  Custodian scans or types each part number     DRAFT — no locks held
          → system shows description, on-hand, location
Step 3  Custodian selects equipment + purpose         DRAFT (per-line override available)
Step 4  Custodian reviews the complete cart           DRAFT
          → resulting balance per line
          → warnings: insufficient stock, falls below minimum after this withdrawal
Step 5  Personnel scans badge                         → resolve by badge_uid_hash
          → display NAME and PHOTO for the custodian's visual confirmation
Step 6  Custodian confirms                            ┐
Step 7  Automatic deduction                           ├─ ONE SHORT DATABASE TRANSACTION
Step 8  Transaction logged                            ┘
```

The committed step, precisely:

```
BEGIN
  SELECT part_id, on_hand FROM part_stock
   WHERE part_id IN (:ids)
   ORDER BY part_id          -- consistent lock ordering; see 5.2
     FOR UPDATE

  for each line:
      re-validate quantity against the freshly locked on_hand
      INSERT stock_movements (quantity = -qty, balance_after = on_hand - qty, ...)
      UPDATE part_stock SET on_hand = on_hand - qty

  UPDATE stock_transactions SET status='completed', recorded_at=now()
  INSERT audit_logs
COMMIT

-- outside the transaction:
dispatch AlertEvaluationJob(affected part ids)
```

5.2 The two details that carry the correctness of this system

Consistent lock ordering. Locks are acquired with `ORDER BY part_id` in a single statement before any update. Two multi-line transactions touching an overlapping set of parts in different orders would otherwise deadlock. Today, with one station, this is invisible; the moment a second station is added it becomes a production incident. Ordering costs nothing to do now.

Short transactions. Every lookup, validation and human review happens *outside* the transaction. The lock is held for the duration of a handful of inserts, not for however long the custodian takes to review the cart. Target: **under 500 ms at p95 for a ten-line withdrawal** — a person is standing at a counter waiting.

Atomicity is all-or-nothing. If any line fails validation, the entire transaction is rolled back with a clear message naming the offending part. Partial issues are never written.

5.3 Insufficient stock — the override path

Not specified in the proposal. Added deliberately.

The scenario: 03:00, a machine is down, the part is physically on the shelf, but the system reads zero because of an earlier recording error. If the system hard-blocks, the custodian issues the part anyway and writes it on paper — and from that moment the system is no longer trusted.

The design does **not** permit negative stock (`part_stock.on_hand >= 0` is a database constraint). Instead:

1. The custodian requests a supervisor override.

2. The supervisor authenticates and supplies a reason code.
3. The system writes an `adjustment` **movement** (`found_stock` / `unrecorded_receipt`) that corrects the balance — a real, attributed ledger entry.
4. The withdrawal then commits normally against the corrected balance.
5. An `inventory_discrepancy` alert opens, and the event is written to `audit_logs`.

This keeps the ledger honest, keeps the non-negative invariant intact, and records the discrepancy as what it actually is rather than hiding it inside a withdrawal.

The adjustment and the withdrawal commit in a single database transaction. They are two separate `stock_transactions` rows — one `adjustment`, one `withdrawal` — but one atomic commit. If they were committed separately and the withdrawal then failed, the ledger would be left holding an unexplained stock increase that nobody issued, which is precisely the kind of orphan record this whole design exists to prevent.

Sequencing, given that supervisor authorisation is a human step and cannot be performed while holding row locks:

```
Step 4  (review)  custodian hits insufficient stock
                  → supervisor authenticates and supplies a reason code
                  → authorisation is attached to the DRAFT; no locks held yet
Step 6  (commit)  BEGIN
                    SELECT ... FOR UPDATE over the union of affected parts, ORDER BY part_id
                    INSERT adjustment movement   (balance_after = corrected balance)
                    INSERT withdrawal movements  (balance_after = corrected balance - qty)
                    UPDATE part_stock once per part
                  COMMIT
```

The adjusted part is already a member of the withdrawal's part set, so the union is unchanged and the lock ordering of §5.2 is unaffected.

The adjustment quantity is computed under lock at commit, never at authorisation: `adjustment = verified_qty − on_hand_locked`. Computing it at authorisation time is wrong when stock moves in between. Concretely: recorded 0, supervisor verifies 5 on the shelf and authorises; a replenishment of 10 is then committed at another station; at commit the locked balance is 10, and a "bring the balance to 5" adjustment would *remove* 5 units that are physically there. Two rules close this:

- An override adjustment may only **increase** the balance.
- If the locked balance already covers the line, the withdrawal commits **without** the adjustment, the override is discarded, and an `inventory_discrepancy` alert is still opened — the original discrepancy was real and the part needs counting.

5.4 Replenishment (proposal §3B)

Custodian enters part number and quantity received, plus supplier, DR/PO reference and optionally unit cost. Commits through the same locked path with positive quantities. Supplier and reference are not in the proposal but a receiving record without a delivery-receipt number cannot be audited against the supplier, so they are included.

5.5 Reversal and correction

Completed transactions are never edited or deleted. A supervisor issues a **reversing transaction** carrying `reverses_transaction_id` and a reason; it writes equal and opposite movements, and the original moves to status `reversed`. Both remain permanently visible in the ledger.

Two rules keep consumption analytics correct under reversal:

- **Reversing movements keep the original `movement_type`.** A reversed withdrawal writes a `+qty` movement of type `withdrawal`, so the withdrawals-only consumption sum nets to zero with no special case in any report.
- **Reversing movements carry the original transaction's `occurred_at`.** Otherwise a withdrawal on 30 January reversed on 2 February shows +15 consumption in January and −15 in February. Their `balance_after` is still stamped in recording order, per §4.5 — the same treatment as a backdated entry.

Not reversible: `opening` transactions, reversing transactions themselves, and adjustments produced by an override (§5.3) or a count posting (§5.6) — those are corrected by a new adjustment or a count. Reversing a withdrawal that carried an override reverses only the withdrawal; the override adjustment stands, because the stock was physically present.

5.6 Physical counting (fills the §6 gap)

1. **Open session** — scope by location, criticality class, or all parts. `system_qty` is snapshotted per line at open.
2. **Count** — custodian enters counted quantities and `counted_at` is stamped. Withdrawals may continue during the session; movements occurring mid-session are listed on the posting screen so that ordinary issues are not mistaken for shrinkage.
3. **Review** — a `reason_code` is required on every line where a reason is warranted (§5.6.2).
4. **Post** — requires supervisor role. Generates one `count_variance` movement per non-zero `posting_delta` and resolves any open `inventory_discrepancy` alerts for the counted parts.

5.6.1 A physical count corrects against the live balance, not the snapshot

Posting the snapshot difference is wrong, and it breaks the `on_hand >= 0` constraint on a path that is genuinely reachable: a session opens with `system_qty = 5`, three units are legitimately withdrawn while the session is open leaving `on_hand = 2`, the custodian counts an empty shelf and enters 0. The snapshot variance is −5, and posting −5 against an on-hand of 2 drives the balance to −3, which the database rejects with an opaque constraint error at the worst possible moment.

The correct semantics: **a physical count asserts what is on the shelf, so posting must make `on_hand` equal `counted_qty`.** At posting time, inside the locked transaction:

```
posting_delta = counted_qty - on_hand_at_post     (on_hand_at_post read under FOR UPDATE)
```

The resulting balance is exactly `counted_qty`, which is non-negative by its own constraint, so the `on_hand >= 0` check can never be violated by a count posting. `system_qty` and `snapshot_variance` are retained for analysis — they show how far the system had drifted since the session opened — but they are never the posted quantity.

Staleness guard. If any movement for a part occurred after its `counted_at`, the count is stale. Those lines are flagged on the posting screen and require either a re-count or explicit supervisor acknowledgement before the session can be posted.

5.6.2 When a reason code is required

`variance_tolerance_pct` is a percentage while `posting_delta` is absolute, so the comparison needs a denominator — and `system_qty` can be zero (found stock on a part the system believes it has none of), which is a division by zero on the posting screen.

The rule, stated to avoid that:

> A `reason_code` is required when `posting_delta <> 0` **and** either `system_qty = 0` or `abs(posting_delta) / system_qty * 100 > variance_tolerance_pct`.

Any discrepancy on a part the system thought was empty always demands an explanation. The default tolerance of 0 means every non-zero delta requires a reason, which is the safe default to start from and can be relaxed per session once real counting data exists.

5.6.3 Uniqueness under programmatic generation

`stock_transaction_lines` carries `unique (stock_transaction_id, part_id)`. Both programmatic producers respect it: a count session emits exactly one adjustment transaction with one line per part, and `count_lines` is already unique on `(count_session_id, part_id)`; the §5.3 override emits its own separate single-line adjustment transaction. Neither can generate a duplicate.

5.7 Excel import (Phase 1 → Phase 4)

Upload → column mapping → validation → **dry-run preview** (create / update / error counts with per-row errors) → commit. Validates duplicate part numbers, missing descriptions, non-numeric quantities, unknown locations, and level ordering.

Opening balances are written as `opening` movements, never as direct `on_hand` writes. This is what makes the central invariant (§4.1) true from the very first row of imported data.

6. Consumption and depletion analytics

6.1 The baseline formula is the proposal's own

Proposal §5 specifies:

```
Average Monthly Consumption = Total Quantity Withdrawn ÷ Number of Months
```

This is implemented verbatim. No cleverer model is silently substituted. A lead-time-aware reorder point is offered as a documented extension (§6.4), not as a quiet replacement.

6.2 Definitions that must be stated or reports will disagree with each other

- **Consumption counts `withdrawal` movements only.** Adjustments, count variances, opening balances and replenishments are excluded. Without this rule written down, the Monthly Consumption Report and the Depletion Rate Report will produce different numbers for the same part.
- **Window** — trailing 6 months by default, configurable; shortened to the part's actual history if shorter, with a data-sufficiency flag (§6.3).
- `average_daily_consumption (ADC) = total_withdrawn ÷ days_in_window`
- `days_of_cover = on_hand ÷ prediction_rate` — null when the rate is zero
- `projected_reorder_date = today + (on_hand − reorder_point) ÷ prediction_rate`
- `predicted_stockout_date = today + on_hand ÷ prediction_rate` — `prediction_rate` is defined in §6.6

6.3 Data confidence

Every projection is published alongside the number of months of history and the number of discrete withdrawal events behind it.

A part with two movements yields a mathematically valid and operationally meaningless depletion rate. Presenting a confident "reorder on 14 November" derived from two data points is precisely how users learn to stop believing the system. Low-confidence projections are rendered visually distinct and the classification is:

Events in window	Confidence
0	none — no projection shown
1–2	low
3–5	moderate
6+	good

6.4 Movement classification

Fast-moving, slow-moving and dead-stock thresholds are **settings, not constants**, because the right cut-off differs by storeroom and will be tuned during proposal Phase 6.

Ranking is by number of withdrawal events, not quantity withdrawn. Quantities are in different units — ranking by quantity would put 400 metres of cable ahead of the bearing withdrawn every shift, because it adds metres to pieces. Event count is unit-independent.

Defaults, applied in this order, first match wins:

1. **Dead stock** — no withdrawal in 12 months
2. **Slow-moving** — no withdrawal in 90 days
3. **Fast-moving** — at least one withdrawal in the window, and in the top 20% of classified parts by withdrawal event count
4. **Normal** — everything else

A part is **not classified** until 90 days after its first movement. Without this, every part imported at go-live is "slow-moving" on day one.

Deferred to v2: `ROP = (ADC × lead_time_days) + safety_stock`, with safety stock derived from demand variability. `parts.lead_time_days` is captured now so the data exists when that model lands.

6.5 Query strategy

Computed on demand from `stock_movements` via the partial withdrawals index (§4.3). A `part_consumption_daily` rollup table is designed but **not built in v1**; the documented trigger for building it is report p95 latency exceeding 2 seconds. Premature rollups add a staleness class of bug for performance that is not yet needed.

6.6 Predicted out-of-stock

Added at Microtech's request in PRD v1.0. Every figure in §6.2 that divides by a consumption rate uses one **prediction rate**:

```
window_ADC      = withdrawn_in_window ÷ days_in_window
recent_ADC      = withdrawn_last_30d ÷ 30        -- used only with ≥ 3 withdrawal events in the last 30 days
prediction_rate = max(window_ADC, recent_ADC)
predicted_stockout_date = today + on_hand ÷ prediction_rate
```

Why the higher of the two rates. A six-month average reacts slowly: a part whose use doubled last month would be predicted to last twice as long as it actually will. Taking the higher rate makes the prediction err early, which is the safe direction for a stock-out warning. The 3-event floor stops a single large withdrawal from being read as a trend.

Risk is measured against lead time, because a predicted date only matters relative to how long a replenishment takes to arrive:

Risk	Condition
`late`	`predicted_stockout_date < today + lead_time` — an order placed today arrives after the stock-out
`order_now`	`predicted_stockout_date < today + lead_time + warning_buffer`
`ok`	otherwise

`lead_time` is `parts.lead_time_days`, falling back to the `prediction.default_lead_time_days` setting (default 14); `warning_buffer` defaults to 7. This is the first use of lead time; the full lead-time reorder-point model in §6.4 stays deferred.

Evaluated nightly for every active part, not only on commit. The predicted date moves one day closer every day with no transaction at all, so a part can cross into `order_now` overnight. It is also re-evaluated on each commit for the parts affected.

Stated limits, shown in the UI: this is a straight-line projection at the prediction rate. It knows nothing of seasonality, planned shutdowns or scheduled maintenance, and it inherits the §6.3 confidence level — no prediction at `none`, visibly marked at `low`. Seasonal forecasting stays in v2.

7. Alerts (proposal §6)

Alert	Condition
Low stock	`on_hand <= min_level` (only when `min_level > 0`)
Critical stock	`on_hand <= critical_level` (only when `critical_level > 0`), **or** `criticality = 'critical'` and `on_hand = 0`
Replenishment	`on_hand <= reorder_point` (only when `reorder_point > 0`), with consumption history in the alert context
High consumption	`consumption_last_30d > (baseline_ADC × 30) × multiplier` (default 2.0) **and** ≥ 3 withdrawal events in the last 30 days **and** ≥ 60 days of baseline history
Inventory discrepancy	A count session marked counted with a variance beyond tolerance, or a §5.3 override. Resolved when the part's count is posted
Predicted out-of-stock	Risk is `order_now` or `late` **and** confidence is `moderate` or `good` (§6.6). Severity `warning`, raised to `critical` when risk is `late` on a part with `criticality = 'critical'`. Resolved when risk returns to `ok` or `on_hand` reaches 0. May be open alongside a stock-level alert — one is about time, the other about quantity

High consumption, precisely. `baseline_ADC` is average daily consumption over the analysis window *excluding the most recent 30 days*. Two errors are avoided: comparing a 30-day total directly against a daily rate (a unit mismatch that fires on almost every active part), and letting the spike being detected inflate its own baseline.

Level thresholds of zero are "not set". Level columns default to 0, so without the `> 0` guard every unconfigured part at zero stock would raise three alerts the moment the Excel import lands. The exception is deliberate: a part classified critical at zero stock always alerts.

At most one stock-level alert per part. Critical, low and replenishment are nested conditions — a part at its critical level satisfies all three. The evaluator keeps only the most severe open; when the balance crosses a level it resolves the current one and opens the new one. History is preserved, and a worsening condition requires fresh acknowledgement.

`critical_level` and `criticality` are separate fields and must stay separate. `critical_level` is a quantity threshold; `criticality` is a classification (critical / important / normal). §3B and §6 use both, and report 7-F depends on the classification while the Critical Stock Alert depends on the threshold. Collapsing them breaks one or the other.

7.1 Alerts are state-based, not event-based

An alert **opens once** when its condition becomes true, remains open until acknowledged or resolved, and **auto-resolves** when the condition clears. It does not re-fire on every transaction. The partial unique index in §4.3 enforces this at the database level, so the evaluator is safe to run on every single commit.

This is the difference between an alerting feature that is used and one that is filtered away. After three days of forty duplicate emails, people build a mail rule and the feature is dead.

7.2 Evaluation and delivery

Evaluated (a) after each committed transaction, for affected parts only, and (b) in a nightly sweep across all active parts, which catches time-based conditions no transaction would trigger.

Delivery: in-app dashboard and badge counts (live), plus a daily email digest to supervisors and planners. Per-alert instant email is available but off by default, for the reason above.

8. Security and operational risk

8.1 RFID provides accountability, not prevention

Proposal §2 states an objective to "prevent unauthorized withdrawal of spare parts," and §10 repeats it as "Reduced Unauthorized Withdrawals."

Stated plainly: **low-frequency 125 kHz proximity cards are trivially cloneable** with inexpensive readily-available hardware. What the badge scan genuinely delivers is **attribution** — a complete, permanent, per-person record of who received which parts — which is a strong deterrent and the foundation of report §7G. The actual access control in this system is the custodian physically standing at the counter.

This should be worded accurately in client-facing material so that the control is not over-sold. **If badges are being purchased new, specify 13.56 MHz MIFARE DESFire**, which supports mutual authentication and is not clonable by the same trivial means.

8.2 Badge storage

Badge UIDs are stored as `sha256(uid || pepper)` for scan lookup, plus a Laravel-encrypted copy for administrative use, plus the last four characters for display. Reports render them masked as `A1234****`, matching the proposal's own §3B sample. A database compromise does not yield a usable set of badge identifiers.

8.3 Application security baseline

TLS everywhere; Laravel CSRF and policy enforcement on every action; rate limiting on badge lookup to prevent enumeration; credentials only in server-side environment configuration; deny-all RLS as described in §3.3; `audit_logs` capturing every configuration change.

8.4 Availability: online-only against a 24/7 storeroom

This is the most significant operational risk in the design. The system is cloud-hosted; the storeroom runs 24/7. When the internet link or Supabase is unavailable, parts cannot be issued — potentially during exactly the 03:00 breakdown that proposal §10 says the system exists to prevent. The Supabase free tier carries no SLA, which sharpens this.

Mitigations, **in v1 scope**:

1. **Paper fallback form** — a printed, numbered form kept at the station, capturing the same fields.
2. **Backdated entry mode** — transactions entered afterwards with the true `occurred_at`, the actual `recorded_at`, `is_backdated = true` and a mandatory reason. Ledger balances stay correct and the gap is visible rather than hidden.
3. **Draft resilience** — the withdrawal cart is persisted so a brief connectivity blip does not discard a half-entered transaction.

v2 path: a PWA station with an offline queue, which is the point at which the Inertia/SPA architecture (§3.2) becomes worth its cost.

9. Reports (proposal §7 A–H)

All eleven (A–K), each implemented as a filtered query plus a Livewire view plus CSV/XLSX and PDF export, over one shared filter component and one shared export pipeline.

	Report	Primary source	Key filters
A	Current Inventory	`parts` ⋈ `part_stock`	location, criticality, status
B	Spare Parts Withdrawal	`stock_movements` (withdrawals)	date range, part, personnel, equipment, purpose
C	Monthly Consumption	withdrawals grouped by month	date range, part
D	Depletion Rate	`ConsumptionCalculator`	criticality, confidence level
E	Low Stock	`part_stock` vs `min_level`	location, criticality
F	Critical Spare Parts	`parts.criticality = 'critical'`	availability status
G	Personnel Withdrawal History	withdrawals by `personnel_id`	personnel, date range
H	Equipment Consumption	withdrawals by `equipment_id`	equipment, date range
I	**Replenishment Report**	`stock_movements` (replenishments)	date range, part, supplier, reference
J	**Exceptions Report**	`stock_transactions` flagged backdated / manual identification / override, plus reversals and adjustments	date range, exception type, user
K	**Predicted Out-of-Stock Report**	`ConsumptionCalculator` prediction (§6.6), sorted by predicted date	risk, criticality, location, predicted date range

Report J is an addition: overrides, manual identifications and backdated entries each bypass part of the normal control, so supervisors need one place to review all of them.

9.1 Report I is not in §7 — and needs to be

The proposal's §3B function list requires "Replenishment reports", but §7 enumerates only reports A through H and contains no replenishment report. That is an internal inconsistency in the proposal. Since §3B also requires "Inventory replenishment history" as a stored field and §5 asks for replenishment planning, the omission from §7 reads as an oversight rather than a decision, so Report I is included. **Worth confirming with Engr. Agaya.**

9.2 Dashboard (§3B, "Dashboard/summary view")

A single landing screen, and the only screen most managers will open: open alerts by severity; parts below reorder point; parts below critical level; today's withdrawal and replenishment counts; the last ten transactions; and top consumed parts for the current month. Every tile links through to the report behind it.

Exports above a row threshold are generated on the queue and delivered as a signed Supabase Storage link, so that a large report cannot fail against the request timeout.

10. Hardware integration boundary

The reader is abstracted behind a single interface:

```
interface BadgeReader {
    public function normalise(string $rawInput): string;
}
```

Assumed implementation: HID keyboard-wedge. The reader presents as a USB keyboard, "types" the card UID and sends Enter. This needs no drivers, works identically in any browser, and reduces the scan to a focused input field with a submit handler. It is also the cheapest and most widely available class of reader.

If a serial or vendor-SDK reader is chosen instead, a small local agent process is required on the station to bridge it to the browser. Only the `BadgeReader` implementation changes — no business logic is affected.

Procurement recommendation: HID keyboard-wedge, 13.56 MHz MIFARE DESFire (per §8.1).

Other hardware per proposal §8: workstation, UPS (essential — an interrupted commit is survivable because transactions are atomic, but an unplanned shutdown mid-count is not), network connection, and labels for storage locations.

11. Supabase free tier: constraints and mitigations

Microtech has selected the free tier for its own two installs, production and demo. Each customer install is hosted per customer (§3.6, PRD Appendix A Q17). Limits below were verified against Supabase's pricing page and platform docs on 2026-09-13.

Constraint	Impact	Mitigation (v1 scope)
No automated backups	Highest risk on the tier. This is a system of record.	**Nightly `pg_dump` via the Laravel scheduler to an off-platform destination** (office NAS, Google Drive or S3 — *not* Supabase Storage, which shares the failure domain). Retain 30 daily and 12 monthly. Restore drill during Phase 4 UAT.
Project pauses after 1 week of low activity	Daily storeroom use keeps production active. **The demo install is the real exposure**: it can sit unused between customer demonstrations and be paused exactly when it is needed.	Daily scheduler heartbeat on both installs — Supabase states a few database requests a day is enough. A paused project can be resumed from the dashboard within 90 days.
2 active free projects	Microtech production and demo use the whole allowance. A staging environment, or hosting a customer install on Microtech's account, requires Pro (from $25/month, which includes 7 days of daily backups).	Develop and test against local Postgres, which does not count. Decide customer hosting before the first customer install.
Lower connection limits	Direct connections exhaust quickly.	Transaction-mode pooler, mandatory (§12.1).
No SLA	Sharpens the §8.4 availability risk.	Paper fallback and backdated entry, already in scope.
500 MB database	**Not a practical constraint.** At 50 withdrawal lines/day the ledger grows roughly 6 MB/year including indexes; even at 500 lines/day there is multi-year headroom.	Monitor; revisit only if volume is an order of magnitude above estimate.
1 GB storage, 5 GB egress/month	Part photos and generated exports accumulate.	Exports expire after 7 days; part photos resized on upload.

Upgrade trigger: if the restore drill in Phase 4 shows an unacceptable recovery point, or if the business decides it cannot tolerate self-managed backups, the Pro tier with daily backups and PITR is the remedy. The decision is theirs; the design functions on free tier with the `pg_dump` job in place.

12. Non-functional requirements

	Target
Withdrawal commit latency	< 500 ms p95, ten-line transaction
Part number lookup	< 150 ms p95
Report page load	< 2 s p95 (rollup trigger, §6.5)
Concurrency	Correct under simultaneous stations from day one; deadlock-free by lock ordering
Retention	Ledger retained indefinitely; never deleted
Recovery point	≤ 24 h via nightly dump (free tier); improves to minutes on Pro + PITR
Browser support	Current Chrome and Edge
Accessibility	Keyboard-operable throughout — the entire withdrawal flow must be completable without a mouse, because the custodian's hands are on a scanner and a keyboard, not a trackpad

12.1 Laravel + Supabase connection configuration

Supabase's **transaction-mode pooler (port 6543) breaks Laravel's default PDO prepared statements** — a named prepared statement created on one pooled connection will not exist on the next. This surfaces as intermittent `prepared statement "..." does not exist` errors under load, typically first seen in staging.

Resolution, pinned here so it is not rediscovered mid-project:

```
// config/database.php — pgsql connection
'options' => [
    PDO::ATTR_EMULATE_PREPARES => true,
],
```

Use session mode (port 5432) only for migrations and the backup job.

12.2 Generated columns in Laravel migrations

`count_lines.snapshot_variance` is a generated column. Laravel's schema builder expresses this as:

```
$table->decimal('snapshot_variance', 14, 3)
      ->storedAs('counted_qty - system_qty')
      ->nullable();
```

Use `storedAs()`, not `virtualAs()` — Postgres implements only `GENERATED ALWAYS AS ... STORED` and rejects virtual generated columns, so `virtualAs()` fails at migration time rather than at review.

13. Testing strategy

Pest, with a real Postgres instance in CI rather than SQLite — the design depends on `FOR UPDATE`, partial indexes, generated columns and triggers, none of which SQLite reproduces.

13.1 The invariant test

The single most valuable test in this system: generate a randomised sequence of withdrawals, replenishments, adjustments, reversals, count postings **and backdated entries**, then assert for every part that `part_stock.on_hand == SUM(stock_movements.quantity)` and that each `balance_after` equals the running sum of all movements for that part with a lower `id` — **ordered by `id`, not by `occurred_at`**, per §4.5. Including backdated entries in the generated sequence is the point: an `occurred_at` ordering would pass on well-behaved data and fail the moment the paper fallback is used in production. If this test passes, the ledger is sound.

A companion test asserts the count-posting guarantee of §5.6.1 directly: after posting any count session, every counted part's `on_hand` equals its `counted_qty` exactly, including the case where withdrawals occurred while the session was open.

13.2 Concurrency test

Two parallel database transactions withdrawing the same part must not oversell it, and a pair of multi-line transactions over an overlapping part set, committed simultaneously, must not deadlock.

13.3 Scheduled verification in production

A nightly job re-derives every part's balance from the ledger and compares it to `part_stock`, raising an operational alert on any divergence. This proves the invariant continuously against real data, not merely in CI.

13.4 Remaining coverage

- **Unit** — `ConsumptionCalculator`, depletion projection, confidence classification and each alert rule are pure functions, tested table-driven across boundary cases (zero consumption, single event, window shorter than history).
- **Feature** — commit atomicity, insufficient-stock rejection, the override path, void/reverse, count posting, import dry-run correctness, RBAC (each role against each action).
- **End-to-end** — the full scan-last withdrawal flow in a browser, simulating the keyboard-wedge reader as keystrokes terminated by Enter.

13.5 Install tests

These prove the §3.6 rules hold, so a customer install never depends on manual database work:

- **Fresh install** — CI creates an empty plain Postgres database (not Supabase), runs all migrations and the default seed, creates an administrator, and runs the smoke suite. If this passes, the provisioning checklist works without hand-written SQL.
- **Seeder guard** — the sample-data seeder aborts when `APP_INSTALL_TYPE=production`.
- **No hard-coded company values** — a test switches time zone, currency, transaction prefixes and the purpose list through settings, then asserts that transaction numbers, consumption periods and report headers follow them.

14. Delivery phases (mapped to proposal §11)

Proposal phase	Engineering work
1 — Inventory assessment	Excel data audit; agree the CSV import contract; min/max and criticality workshop; equipment register; badge technology confirmed
2 — System development	**M1** schema, ledger, auth, RBAC · **M2** withdrawal flow end to end · **M3** replenishment, adjustments, importer · **M4** analytics, alerts, dashboard · **M5** the eleven reports (A–K) and exports · **M6** counting, backup job, backdated entry
3 — Hardware installation	Reader procurement and acceptance test; workstation and UPS; location labelling; network
4 — Data migration and testing	Import with dry-run; full physical verification count; UAT against §4 and §7; **backup restore drill**; concurrency and invariant suites green
5 — Implementation and training	Custodian training (all shifts); requestor briefing; **two-week parallel run with paper**; go-live
6 — Continuous improvement	Threshold tuning from real consumption; lead-time reorder model; rollup tables if warranted; offline PWA evaluation
Each customer install	Repeat Phases 1, 3, 4 and 5 for that customer using the §3.6 provisioning checklist. No development phase unless a request is accepted as a product feature for every install

Phases 1–6 run first for Microtech's production install. The demo install is provisioned alongside it in Phase 4 and loaded with sample data rather than real data.

15. Traceability — every proposal requirement

Proposal	Requirement	Where it lands
§2	All twelve objectives	Covered across §4–§9; accountability enforced by the `completed_withdrawal_is_attributed` constraint (§4.3)
§3A	Custodian duties	`custodian` role (§3.4); counting (§5.6); replenishment (§5.4)
§3B	All 12 database fields	`parts`, `part_stock`, `stock_movements` (§4.3). "Equipment/application" → `part_equipment`; "Depletion rate" and the two history fields are derived from the ledger, not stored columns
§3B	All 19 software functions	§5 workflows, §6 analytics, §7 alerts, §9 reports (incl. Report I and the dashboard)
§3B	Audit trail with running balance	`stock_movements.balance_after` (§4.1, §4.6)
§3C	Scanner for identification only, scanned last	§5.1 Step 5; §10
§4	Withdrawal Steps 1–8	§5.1, one-to-one mapping
§4 Step 8	All 10 transaction record fields	`stock_transactions` + lines + movements (§4.3)
§5	Consumption and depletion analysis	§6
§5	`Avg Monthly Consumption = Qty ÷ Months`	§6.1, implemented verbatim
§5	Min/max, safety stock, reorder point	§4.4, with the ambiguity between them resolved
§6	5 alert types	§7
§7	Reports A–H	§9, all eight, **plus Report I** for the §3B replenishment report §7 omits (§9.1) **Report J** for exceptions and **Report K** for predicted out-of-stock
§11 Phase 1	"Classify parts according to equipment"	`part_equipment` (§4.3)
§8	Hardware	§10
§9	Architecture flow	§3.1, §5.1
§10	Expected benefits	Outcomes of §4–§9; §8.1 qualifies the "prevent unauthorized withdrawal" claim
§11	Phases 1–6	§14

15.1 Additions beyond the proposal — require client confirmation

Addition	Rationale
Physical/cycle counting subsystem (§5.6)	§6 promises an Inventory Discrepancy Alert that nothing in the proposal can produce
Supervisor override via adjustment (§5.3)	Without it, the system is bypassed on its first bad night
Paper fallback and backdated entry (§8.4)	Online-only system against a 24/7 operation with no SLA
Nightly off-platform backup (§11)	Free tier has no automated backups
Void by reversing transaction (§5.5)	Mistakes are certain; deletion would destroy the audit trail
Unit of measure on parts	Not in §3B, but oil and cable cannot be counted in pieces

Addition	Rationale
Supplier and DR/PO on replenishment (§5.4)	A receipt without a reference number is unauditable
Data confidence on projections (§6.3)	Protects user trust in the depletion figures
Report I — Replenishment (§9.1)	§3B requires it; §7 omits it. Reads as an oversight in the proposal
`part_equipment` association (§4.3)	§3B's "Equipment/application" part field and Phase 1's "classify parts according to equipment"; a part usually fits several machines
Stock level semantics defined (§4.4)	§3B and §5 use min level and reorder point without distinguishing them; undefined, two alerts would duplicate each other

16. Open questions for the client

> **Superseded by PRD Appendix A** (`docs/prd/Microtech_SPIMS_PRD_v1.0.md`), which carries these ten questions forward and adds eight more — supervisor availability on night shifts, manual identification, returns of unused parts, immediate critical-stock email, application hosting, success-measure targets, Microtech's starting parts list, default hosting for customer installs, and use of the customer's Excel list — split into Microtech installs (current focus) and parked customer questions, each tagged with what it blocks. None blocks development. Maintain the list there.

1. **Badge technology** — are existing Microtech employee IDs to be reused, and if so are they 125 kHz proximity, 13.56 MHz MIFARE, or barcode? Determines reader procurement (§10) and how §8.1 is worded.
2. **Counting cadence** — how often, and at what scope? Drives the Phase 4 verification plan.
3. **Backup destination** — which off-platform location, and who holds the credentials? (§11)
4. **Report distribution** — who receives the daily alert digest, and does management want a scheduled weekly or monthly report by email?
5. **Purpose codes** — is the §4 Step 3 list final, or does Microtech have existing maintenance work-type codes to align with?
6. **Equipment register** — does a machine list exist to import, or is it built during Phase 1? Report §7H depends on it.
7. **Number of shifts and custodians** — for user account provisioning and training scheduling.
8. **Stock level semantics** — confirm the §4.4 definitions of reorder point vs minimum vs critical level. If Microtech treats minimum and reorder point as the same thing, the Low Stock and Replenishment alerts should be merged rather than both fired.
9. **Replenishment report** — confirm that its absence from §7 was an oversight and Report I (§9.1) is wanted.
10. **Part-to-equipment classification** — does one part typically serve several machines? The `part_equipment` design assumes yes. If each part maps to exactly one machine, this simplifies.

17. Decisions recorded

Decision	Choice	Date
UI architecture	Laravel + Livewire 3 (not Inertia/SPA)	2026-09-12
Authentication	Laravel Fortify; Supabase Auth not used	2026-09-12
Database	Supabase Postgres, **free tier**	2026-09-12
Stock model	Append-only ledger with derived cache	2026-09-12
Counting subsystem	In scope for v1	2026-09-12
Insufficient-stock override	In scope, via attributed adjustment	2026-09-12
Paper fallback / backdated entry	In scope for v1	2026-09-12
Backups	Self-managed nightly `pg_dump` off-platform	2026-09-12
Status vocabulary	`draft / completed / cancelled / reversed`, aligned to the PRD	2026-09-13
Manual personnel identification	Supervisor-authorised fallback when a badge is missing or the reader fails; pending client confirmation	2026-09-13
Movement class ranking	By withdrawal event count, not quantity (units are not comparable)	2026-09-13
Reversal dating	Reversing movements keep original type and `occurred_at`	2026-09-13
Deployment model	One install per company (separate application and database); no shared multi-company hosting	2026-09-13
Microtech installs	Production (own storeroom) and demo (sample data), kept separate	2026-09-13
Withdrawal purposes	Per-install table seeded with the proposal's list, replacing the `CHECK` constraint	2026-09-13
Predicted out-of-stock	Straight-line projection at max(window, recent 30-day) rate; risk against lead time; evaluated nightly and on commit	2026-09-13